

리눅스 3.5 - Red-Black Tree

남궁명수

자료구조:

- 각 노드에 Red 또는 Black 색상 정보를 추가한 균형 이진 탐색 트리

노드 정보:

- 키
- 데이터 또는 데이터 참조
- 부모 포인터
- 왼쪽·오른쪽 자식 포인터
- 색상 정보

핵심 규칙:

- 모든 노드는 Red 또는 Black
- 루트는 Black
- NIL 리프는 Black
- Red 노드의 자식은 Black
- 모든 NIL 리프까지의 경로에 같은 수의 Black 노드 존재

탐색:

- 일반 이진 탐색 트리와 동일하게 키의 대소 관계로 이동
- 색상은 탐색 방향이 아니라 균형 유지에 사용

삽입:

- 새로운 노드를 Red로 삽입
- Red-Red 위반 발생 시 색상 변경과 회전으로 복구

삭제:

- Black 노드 삭제 시 Black Height가 깨질 수 있음
- 형제 노드의 상태에 따라 색상 변경과 회전으로 복구

시간 복잡도:

- 탐색: $O(\log N)$
- 삽입: $O(\log N)$
- 삭제: $O(\log N)$

주요 사용 위치:

- 프로그램과 운영체제 커널의 RAM 내부 자료구조

데이터베이스 인덱스에 주로 사용하지 않는 이유:

- 자식이 최대 2개라 Fan-out이 작음
- B+Tree보다 트리 높이가 커짐
- 저장장치 페이지 I/O가 많아질 수 있음
- 데이터베이스의 페이지 단위 저장 방식에는 B+Tree가 더 적합

[Red-Black Tree]

Red-Black Tree는 일반 이진 탐색 트리(BST)에 색상 규칙을 추가하여 트리가 한쪽으로 지나치게 치우는 것을 방지한 균형 이진 탐색 트리다.

각 노드는 다음 정보를 가진다.

- 탐색에 사용하는 키
- 데이터 또는 데이터 참조
- 왼쪽 자식 포인터
- 오른쪽 자식 포인터
- 부모 포인터
- Red 또는 Black 색상

```
type Node struct {
    Key    int
    Color  Color
    Left   *Node
    Right  *Node
    Parent *Node
}
```

이름은 노드를 실제로 빨간색과 검은색으로 저장해서 붙은 것이 아니라, 각 노드에 Red 또는 Black 상태를 나타내는 정보를 저장하기 때문에 붙은 이름이다.

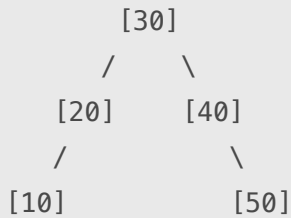
Red-Black Tree는 색상 규칙과 회전을 사용해 트리의 높이를 $O(\log N)$ 으로 제한한다.
따라서 탐색, 삽입, 삭제의 최악 시간 복잡도가 모두 $O(\log N)$ 이다.

[1] 이진 탐색 트리의 문제

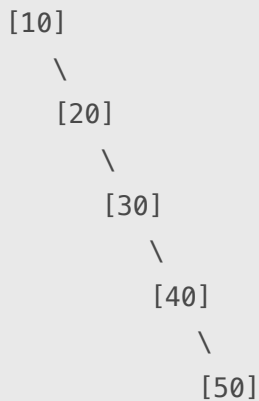
일반 이진 탐색 트리는 다음 규칙을 가진다.

왼쪽 서브트리의 키 < 부모 키 < 오른쪽 서브트리의 키

트리가 균형 있게 만들어지는 탐색 시간이 $O(\log N)$ 이다.



하지만, 10, 20, 30, 40, 50을 순서대로 삽입하면 다음과 같이 한쪽으로 치우칠 수 있다.



이 경우 50을 찾으려면 모든 노드를 순서대로 확인해야 한다.

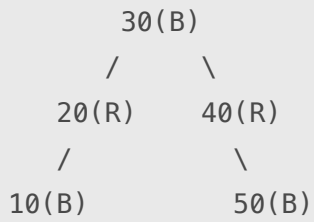
10 → 20 → 30 → 40 → 50

따라서 최악의 탐색 시간이 $O(N)$ 이 된다.

Red-Black Tree는 노드의 색상 규칙을 검사하고, 규칙이 깨지면 색상 변경과 회전을 수행하여 이러한 치우침을 제한한다.

[2] Red-Black Tree의 기본 구조

Red-Black Tree도 이진 탐색 트리이므로 노드 하나가 최대 두 개의 자식을 가진다.



B = Black

R = Red

노드의 검색 순서는 일반 이진 탐색 트리와 같다.

검색 키 < 현재 키

→ 왼쪽 자식으로 이동

검색 키 > 현재 키

→ 오른쪽 자식으로 이동

검색 키 == 현재 키

→ 탐색 완료

Red-Black 색상은 탐색 방향을 결정하지 않는다.

키

→ 탐색 방향 결정

색상

→ 트리의 균형을 유지하는 데 사용

[3] [Red-Black Tree의 규칙]

Red-Black Tree는 다음 규칙을 유지한다.

[규칙 1] 모든 노드는 Red 또는 Black 이다.

Node Color

└ Red

└ Black

모든 노드는 두 색상 중 하나를 가진다.

[규칙 2] 루트 노드는 Black이다.

```
      30(B) ← 루트는 Black
     /    \
    20(R)  40(R)
```

일부 이론에서는 루트 색상이 핵심 균형 조건과 직접 관계없어 생략되기도 하지만, 일반적으로 Red-Black Tree의 규칙을 설명할 때 루트는 Black으로 정의한다.

[규칙 3] 모든 NIL 리프 노드는 Black이다.

Red-Black Tree에서는 실제 자식이 없는 nil 위치도 특별한 NIL 노드로 간주한다.

```
      20(B)
     /    \
    NIL(B) NIL(B)
```

일반 그림에서는 NIL 노드를 생략하지만, 균형 규칙을 검사할 때는 Black노드로 취급한다.

실제로 그리는 모습:

```
      20(B)
```

규칙상 전체 모습:

```
      20(B)
     /    \
    NIL(B) NIL(B)
```

[규칙 4] Red 노드의 자식은 반드시 Black이다.

Red 노드 아래에는 Red 노드가 연속해서 올 수 없다.

가능:

```
      30(B)
     /
    20(R)
   /
  10(B)
```

불가능:

```
      30(B)
      /
     20(R)
      /
    10(R) ← Red가 연속됨
```

이를 다음과 같이 표현할 수 있다.

Red 노드

```
|— 왼쪽 자식: Black
|— 오른쪽 자식: Black
```

부모와 자식이 모두 Red인 상태를 Red-Red 위반이라고 한다.

[규칙 5] 한 노드에서 모든 NIL 리프까지의 Black 노드 수가 같다.

어떤 노드를 기준으로 아래쪽의 모든 NIL 리프까지 내려가는 경로에는 같은 수의 Black 노드가 존재해야 한다.

이때 경로에 존재하는 Black 노드의 수를 Black Height라고 한다.

```
      30(B)
     /  \
    20(R) 40(B)
   /      \
  10(B)    50(R)
```

각 경로에서 만나는 Black 노드의 수가 같도록 유지되어야 한다.

Black Height 규칙 때문에 한쪽 경로만 무한히 길어지는 것을 막을 수 있다.

[4] 색상 규칙이 균형을 유지하는 원리

Red-Black Tree는 왼쪽과 오른쪽 서브트리의 높이를 완전히 같게 만들지는 않는다.

대신 다음 두 규칙이 높이 차이를 제한한다.

1. Red 노드가 연속해서 나올 수 없다.
2. 모든 경로의 Black 노드 수가 같다.

가장 짧은 경로가 모두 Black 노드로 구성되어 있다고 해보자.

Black → Black → Black

가장 긴 경로에는 Black 노드 사이마다 Red가 들어갈 수 있다.

Black → Red → Black → Red → Black

하지만 Red노드는 연속될 수 없으므로 가장 긴 경로도 가장 짧은 경로의 두 배를 넘을 수 없다.

가장 긴 경로 ≤ 가장 짧은 경로 × 2

따라서 트리의 높이가 $O(\log N)$ 으로 제한한다.

정확히는 노드가 N개인 Red-Black Tree의 높이는 최대 다음 범위로 제한된다.

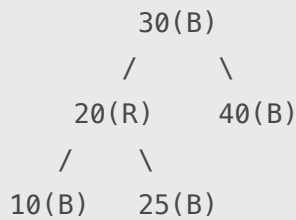
높이 ≤ $2\log_2(N + 1)$

Red-Black Tree는 완벽하게 균형 잡힌 트리는 아니지만, 한쪽으로 심하게 치우치지 않는 충분한 균형을 유지한다.

[5] Red-Black Tree의 탐색

Red-Black Tree의 탐색 방법은 일반 이진 탐색 트리와 같다.

다음 트리에는 25를 찾는다고 해보자.



탐색 과정은 다음과 같다.

1. 25와 루트 30을 비교한다.
2. $25 < 30$ 이므로 왼쪽으로 이동한다.
3. 25와 20을 비교한다.
4. $25 > 20$ 이므로 오른쪽으로 이동한다.
5. 키 25를 발견한다.

색상은 탐색에 직접 사용되지 않는다.

탐색

→ 키의 대소 관계만 사용

삽입·삭제 후 균형 복구

→ 색상 정보 사용

트리의 높이가 $O(\log N)$ 이므로 탐색 시간도 다음과 같다.

$O(\log N)$

[6] 회전 연산

회전(Rotation)은 이진 탐색 트리의 정렬 순서를 유지하면서 부모와 자식의 위치를 변경하는 연산이다. Red-Black Tree는 회전을 통해 한쪽으로 치우친 구조를 조정한다.

회전에는 다음 두 종류가 있다.

- Left Rotation
- Right Rotation

회전하더라도 이진 탐색 트리의 정렬 규칙은 유지된다.

[Left Rotation]

다음 구조에서 20을 기준으로 왼쪽 회전한다고 해보자.

회전 전:

```
  20
   \
   30
    \
    40
```

30이 위로 올라가고 20이 왼쪽 자식이 된다.

회전 후:

```
   30
  /  \
 20   40
```

키의 정렬 순서는 그대로다.

$20 < 30 < 40$

왼쪽 회전은 오른쪽으로 치우친 구조를 조정할 때 사용한다.

[Right Rotation]

다음 구조에서 30을 기준으로 오른쪽 회전한다고 해보자.

회전 전:

```
      30
     /
    20
   /
  10
```

20이 위로 올라가고 30이 오른쪽 자식이 된다.

회전 후:

```
      20
     /  \
    10   30
```

키의 정렬 순서는 그대로다.

$10 < 20 < 30$

오른쪽 회전은 왼쪽으로 치우친 구조를 조정할 때 사용한다.

[회전의 핵심]

회전은 키의 정렬 순서를 변경하지 않고 트리의 높이와 부모-자식 관계를 조정한다.

회전 전 중위 순회:

$10 \rightarrow 20 \rightarrow 30$

회전 후 중위 순회:

$10 \rightarrow 20 \rightarrow 30$

따라서 이진 탐색 트리의 검색 규칙은 그대로 유지된다.

회전 자체는 몇 개의 포인터만 변경하므로 시간 복잡도는 다음과 같다.

$O(1)$

[7] Red-Black Tree의 삽입

[기본 삽입 과정]

새로운 키는 먼저 일반 이진 탐색 트리의 삽입 규칙에 따라 추가된다.
새 노드는 일반적으로 Red로 삽입한다.

새 노드
→ Red로 삽입

Black으로 삽입하면 해당 경로의 Black Height가 즉시 증가할 수 있기 때문이다.
Red로 삽입하면 Black Height는 변하지 않는다.

다만 부모가 Red라면 Red-Red 위반이 발생할 수 있다.

1. BST 규칙에 따라 삽입 위치 탐색
2. 새 노드를 Red로 삽입
3. Red-Black 규칙 검사
4. 규칙 위반 시 색상 변경 또는 회전
5. 루트를 Black으로 설정

[부모가 Black인 경우]

새 노드를 Red로 삽입했지만 부모가 Black이라면 Red-Red 위반이 없다.

삽입 전:
20(B)

30 삽입:
20(B)
 \
 30(R)

추가 조정이 필요하지 않다.

[부모와 삼촌이 모두 Red인 경우]

다음 상황에서 새 노드 10이 삽입되었다고 해보자.

```
      30(B)
     /   \  
  20(R) 40(R)
   /
 10(R)
```

부모 20과 새 노드 10이 모두 Red이므로 규칙을 위반한다.

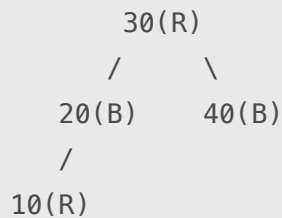
삼촌 40도 Red이므로 색상을 변경한다.

부모 20 → Black

삼촌 40 → Black

조부모 30 → Red

색상 변경 후:

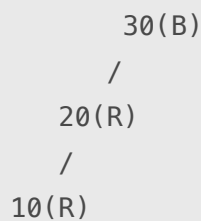


이를 Recoloring이라고 한다.

[부모는 Red이고 삼촌은 Black인 경우]

삼촌이 Black인 경우에는 회전과 색상 변경이 필요할 수 있다.

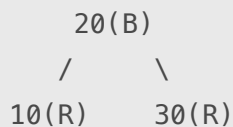
삽입 전:



20과 10이 연속된 Red이므로 규칙을 위반한다.

조부모 30을 기준으로 오른쪽으로 회전한다.

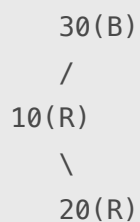
회전 및 색상 변경 후:



트리의 정렬 순서와 Red-Black 규칙이 다시 복구된다.

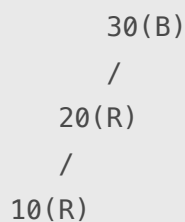
삽입 위치가 꺾인 형태라면 먼저 부모를 회전하여 일직선 형태로 만든 다음 조부모를 반대 방향으로 회전한다.

회전 전

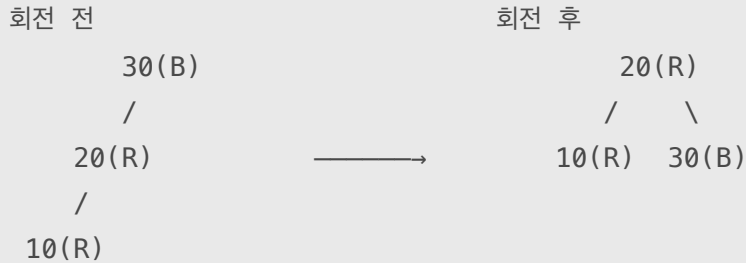


→

회전 후



1. 부모 10을 기준으로 왼쪽 회전
2. 조부모 30을 기준으로 오른쪽 회전
3. 색상 변경



[삽입 시간 복잡도]

삽입 위치를 찾는데 $O(\log N)$ 이 필요하다.

균형을 복구하는 과정에서 색상 변경이 루트 방향으로 전파될 수 있지만, 트리 높이가 $O(\log N)$ 이므로 전체 삽입 시간은 다음과 같다.

삽입: $O(\log N)$

회전 한 번의 비용은 $O(1)$ 이다.

[8] Red-Black Tree의 삭제

[기본 삭제 과정]

삭제도 먼저 일반 이진 탐색 트리의 규칙에 따라 수행한다.

1. 삭제할 키 탐색
2. BST 규칙에 따라 노드 삭제
3. 삭제된 노드의 색상 확인
4. Red-Black 규칙 위반 여부 검사
5. 색상 변경과 회전으로 균형 복구

Red 노드를 삭제하면 Black Height가 변하지 않으므로 비교적 간단하다.

Red 노드 삭제

- Black Height 변화 없음
- 추가 조정이 필요하지 않을 수 있음

Black 노드를 삭제하면 해당 경로의 Black 노드 수가 줄어들 수 있다.

Black 노드 삭제

- 특정 경로의 Black Height 감소
- 균형 복구 필요

삭제 과정에서는 부족해진 Black을 개념적으로 Double Black 상태라고 표현하기도 한다.

[삭제 후 균형 복구]

Black 노드를 삭제한 후에는 형제 노드와 형제의 자식 색상을 확인한다.

상황에 따라 다음 작업을 수행한다.

- 형제 노드의 색상 변경
- 부모 노드의 색상 변경
- 형제의 자식 노드 색상 변경
- Left Rotation
- Right Rotation
- Double Black을 부모 쪽으로 이동

삭제 복구는 삽입 복구보다 경우의 수가 많고 복잡하다.

그러나 복구 작업은 트리 높이 범위 안에서만 발생하므로 삭제의 전체 시간 복잡도는 다음과 같다.

삭제: $O(\log N)$

[9] Red-Black Tree의 저장 위치

[Red-Black Tree는 주로 RAM에서 사용한다]

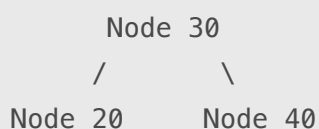
Red-Black Tree는 일반적으로 프로그램이나 운영체제 커널이 RAM에 구성하여 사용하는 자료구조다.

각 노드는 포인터를 통해 다른 노드와 연결된다.

RAM

Node 30

- ├ Left 포인터 → Node 20의 메모리 주소
- └ Right 포인터 → Node 40의 메모리 주소



CPU는 RAM에 저장된 노드의 키와 포인터를 읽으며 탐색한다.

CPU

↓

RAM의 현재 노드 읽기

↓

키 비교

↓

왼쪽 또는 오른쪽 포인터 확인

↓

다음 노드 접근

Red-Black Tree의 포인터는 일반적으로 같은 프로세스의 가상 주소 공간에 있는 노드의 메모리 주소다.

반면 데이터베이스의 B+Tree에서 사용하는 자식 포인터는 단순한 RAM 주소라기보다 DBMS가 관리하는 인덱스 페이지 번호나 페이지 식별자를 의미할 수 있다.

구분	Red-Black Tree	DB의 B+Tree
주요 사용 위치	RAM	SSD/HDD에 영구 저장, 필요 시 RAM에 적재
연결 정보	메모리상의 노드 포인터	자식 페이지 번호·페이지 식별자
노드 크기	비교적 작음	DB 페이지 크기에 맞춤
주요 비용	메모리 접근·키 비교	페이지 I/O
자식 수	최대 2개	여러 개

[10] Red-Black Tree의 활용

[대표적인 활용 분야]

Red-Black Tree는 메모리 안에서 정렬된 데이터를 관리하면서 삽입과 삭제가 자주 발생하는 상황에 적합하다.

대표적인 활용 사례는 다음과 같다.

- C++ `std::map`
- C++ `std::set`
- Java `TreeMap`
- Java `TreeSet`
- 운영체제 커널 내부 자료구조
- 메모리 기반 정렬 집합과 맵

구체적인 구현은 언어와 라이브러리 버전에 따라 다를 수 있지만, 정렬된 Map과 Set을 구현하는 대표적인 자료구조로 사용된다.

Red-Black Tree는 다음 연산을 효율적으로 지원한다.

- 특정 키 검색
- 최소·최대 키 탐색
- 이전·다음 키 탐색
- 정렬된 순서로 전체 순회
- 범위에 포함된 키 탐색
- 빈번한 삽입과 삭제

[11] 데이터베이스 인덱스에서 주로 사용하지 않는 이유

[B+Tree보다 Fan-out이 작다]

Red-Black Tree는 이진 트리이므로 노드 하나가 최대 두 개의 자식만 가진다.

Red-Black Tree

- 자식 최대 2개

B+Tree는 노드 하나에 여러 키와 여러 자식 포인터를 가진다.

B+Tree

- 자식 수십~수백 개 가능

따라서 같은 개수의 데이터를 저장해도 Red-Black Tree가 B+Tree보다 높아질 가능성이 크다.

Red-Black Tree

- 높이가 상대적으로 큼
- 더 많은 노드를 따라가야 함

B+Tree

- 높이가 낮음
- 더 적은 페이지를 읽음

[12] 디스크 페이지 구조와 잘 맞지 않는다

데이터베이스는 SSD/HDD에서 데이터를 페이지 단위로 읽는다.
Red-Black Tree의 노드 하나는 키 하나와 소수의 포인터만 저장한다.

Red-Black Tree 노드

[색상 | 키 | 부모 포인터 | 왼쪽 포인터 | 오른쪽 포인터]

페이지 하나를 읽었는데 필요한 노드 정보가 적다면 저장 공간과 I/O를 효율적으로 활용하기 어렵다.

B+Tree는 페이지 하나에 여러 키와 자식 페이지 정보를 저장한다.

B+Tree 내부 페이지

[키 | 자식 포인터 | 키 | 자식 포인터 | ...]

한 번의 페이지 I/O로 많은 키를 비교할 수 있고, 다음에 읽을 자식 페이지 하나를 선택할 수 있다.

B+Tree의 높은 Fan-out

↓

낮은 트리 높이

↓

적은 페이지 I/O

따라서 저장장치 기반 데이터베이스 인덱스에는 일반적으로 B+Tree 계열이 더 적합하다.

[13] AVL Tree와의 차이

AVL Tree도 균형 이진 탐색 트리이지만 Red-Black Tree보다 더 엄격하게 균형을 유지한다.

구분	AVL Tree	Red-Black Tree
균형 기준	왼쪽·오른쪽 높이 차이 제한	색상과 Black Height 규칙
균형 정도	더 엄격함	상대적으로 느슨함
트리 높이	상대적으로 낮음	AVL보다 조금 높을 수 있음
탐색	상대적으로 유리할 수 있음	$O(\log N)$
삽입·삭제	회전이 더 자주 발생할 수 있음	상대적으로 조정 횟수가 적음
적합한 상황	탐색 비중이 매우 높음	삽입·삭제가 자주 발생

[14] B+Tree와의 비교

구분	Red-Black Tree	B+Tree
구조	이진 균형 탐색 트리	다진 균형 탐색 트리
자식 수	최대 2개	여러 개
균형 방식	색상 변경과 회전	분할·재분배·병합
데이터 위치	각 노드	주로 리프 노드
리프 연결	일반적으로 없음	순서대로 연결
트리 높이	상대적으로 높음	낮음
범위 검색	중위 순회로 가능	리프 연결로 효율적
주요 저장 환경	RAM	SSD/HDD 페이지 기반
주요 비용	메모리 접근	저장장치 페이지 I/O
대표 용도	정렬 Map·Set, 커널 자료구조	데이터베이스 인덱스

[15] 시간 복잡도

연산	평균	최악
탐색	$O(\log N)$	$O(\log N)$
삽입	$O(\log N)$	$O(\log N)$
삭제	$O(\log N)$	$O(\log N)$
최소·최대 탐색	$O(\log N)$	$O(\log N)$
회전 1회	$O(1)$	$O(1)$
전체 순회	$O(N)$	$O(N)$